

Real-Time Systems / Real-Time Operating Systems

ECE445M/ECE380L.12, Spring 2024

Midterm Exam Solutions

Date: March 7, 2024

UT EID: _____

Printed Name: _____
Last, First

Your signature is your promise that you have not cheated and will not cheat on this exam, nor will you help others to cheat on this exam:

Signature: _____

Instructions:

- Open book, open notes and open web.
- No electronic devices other than your laptop/PC (cell phones off and stowed away).
- You are allowed to access any resource on the internet, but no electronic communication other than with instructors.
- Please be sure that your answers to all questions (and all supporting work that is required) are contained in the space (boxes) provided.
- For all questions, unless otherwise stated, find the most efficient (time, resources) solution.

Problem 1	20	
Problem 2	20+5	
Problem 3	20	
Problem 4	20+5	
Problem 5	20	
Total	100+10	

Name: _____

Problem 1 (20 points): Miscellaneous

- a) Rank the debugging techniques based on the intrusiveness effect on the program, and explain your reasoning: Breakpoints, Logic Analyzer Pin Toggling, Array Dumps

Breakpoints > Array Dumps > Pin Toggling

Breakpoints stop the flow of execution of the program completely, which breaks any timing.

Array dumps require small code modifications and insertion of a small function call, which affects timing by several cycles (length of the function call)

Pin toggling only requires insertion of 3 instructions (load, xor, store) with minimal impact on execution time/cycles.

- b) How can you detect and prevent stack overflow in your OS?

One possible method is to initialize the stack space with a magic value (e.g. 0x42424242) and check at periodic intervals if the value at the lowest address of the stack space has been overwritten. If so, the top of the stack has reached this point, i.e. the available stack space has been used and the stack likely overflowed (or is about to overflow).

An alternative is to use the hardware support provided by the built-in memory protection unit (MPU) to prevent any accesses to memory regions below the allocated stack space.

- c) Why can't you use spinlock semaphores with a priority scheduler?

In a priority scheduler, any thread calling OS_Wait(), by definition, has to be the thread with the highest priority. If the semaphore is not available and that thread enters the spinning loop, no other thread will ever run to potentially release the semaphore and the system gets stuck.

- d) Why does PendSV need to be the lowest priority (priority = 7)?

We do not want the PendSV_Handler and hence a context switch to occur in the middle of any other interrupt handler. The context switch would otherwise try to switch out a background thread (interrupt handler) and replace it with a regular foreground thread to be switched in, which will either have the wrong stack and crash when trying to return from the interrupt, or not return from the interrupt at all (if LR is replaced), i.e. run the other thread in an interrupt context.

Name: _____

Problem 2 (20+5 points): Blocking Semaphores

Assume a student in the class was trying to implement an OS using a round-robin scheduler with blocking semaphores using the following code, where the run list is implemented as a double linked list (with next and prev pointers in the TCB):

```
void OS_Wait(Sema4Type *semaPt) { long sr;

    sr = StartCritical();

    semaPt->Value--;
    if (semaPt->Value < 0) {
        // Remove from run list
        RunPt->prev->next = RunPt->next;
        RunPt->next->prev = RunPt->prev;

        NextPt = RunPt->next;

        // Move to blocked list
        RunPt->next = semaPt->BlockPt;
        semaPt->BlockPt = RunPt;

        // Context switch
        OS_Suspend(); ContextSwitch();
    }

    EndCritical(sr);
}

void OS_Suspend() {
    NextPt = RunPt->next;
    ContextSwitch();
}
```

```
ContextSwitch
BL PendSV_Handler    LDR R0,=NVIC_INT_CTRL
                        LDR R1,=NVIC_PENDSVSET
                        STR R1,[R0]

                        BX LR

PendSV_Handler
    CPSID      I
    PUSH {R4-R11}
    LDR R0, =RunPt
    LDR R1, [R0]
    STR SP, [R1]
    LDR R1, [R1, #4]    LDR R2,=NextPt
                        LDR R1,[R2]

    STR R1, [R0]
    LDR SP, [R1]
    POP {R4-R11}

    CPSIE      I
    BX LR
```

Name: _____

Identify all the bugs in the code, explain what erroneous behavior will occur with the current implementation and how it could be fixed. Bonus points: extra credit for providing working code and fixes for the bugs by indicating changes to the code above.

Bugs:

1) Race condition between concurrent calls to OS_Wait (a) when decrementing the semaphore value and (b) when updating the BlockPt and RunPt linked list (the latter also has potential race with other methods accessing the BlockPt or RunPt lists, e.g. OS_Signal or OS_AddThread).

-> Solution: make OS_Wait atomic by adding Start/EndCritical

2) ContextSwitch needs to trigger PendSV_Handler, not call it, or else the context switch will not work correctly (would require a different, non-interrupt based context switch routine that is compatible with an interrupt-based switch triggered from a preemptive timer interrupt).

-> Solution: trigger PendSV, don't call the handler directly

3) PendSV_Handler needs to disable interrupts to avoid being interrupted by a higher priority interrupt in the middle of a context switch while we are fiddling with the stack.

-> Solution: disable/enable interrupts at beginning/end of PendSV_Handler

4) Context switch routine assumes RunPt->next to point to the next thread to run, but OS_Wait moves the thread to the BlockPt list before calling OS_Suspend(), which will result in the next blocked thread being switched to.

-> Solution: introduce a global NextPt that determines the thread being switched to, which is set by OS_Wait before updating RunPt->next. Update context switch in PendSV_Handler to use that pointer instead of RunPt->next and make sure that regular context switches also update NextPt (by setting it to RunPt->next in the normal case) before triggering the context switch (e.g. in OS_Suspend(), where other routines such as the SysTick_Handler are assumed to call OS_Suspend()).

Name: _____

Problem 3 (20 points): Synchronization 1

Observe the code below, running with standard blocking semaphores and a strict priority scheduler (lower number is higher priority):

<pre> void task1() { ① OS_Wait(&s1); ④ OS_Wait(&s2); ⑧ printf("t1\n\r"); OS_Signal(&s2); OS_Kill(); } </pre>	<pre> void task2() { ② OS_Wait(&s1); ⑥ printf("t2\n\r"); ... OS_Kill(); } </pre>	<pre> void task3() { ③ OS_Wait(&s2); ③ OS_Signal(&s1); ⑤ OS_Signal(&s1); ⑦ printf("t3\n\r"); ⑦ OS_Signal(&s2); ⑨ OS_Kill(); } </pre>
<pre> Sema4Type s1; Sema4Type s2; void main() { OS_Init(); OS_InitSemaphore(&s1, 0); OS_InitSemaphore(&s2, 1); OS_AddThread(&task1, 128, 1); OS_AddThread(&task2, 128, 2); OS_AddThread(&task3, 128, 3); OS_Launch(); } </pre>		

- a) Indicate the order in which tasks are interleaved in the code above by labeling each line of code in each task with ①, ②, ③, etc. in order of their execution, and show what the terminal output would look like.

Terminal output:

t2
t3
t1

- b) Assume *task2* runs for an excessive amount of time, is this behavior a problem for a real-time system? If so, what problems would it cause and what changes you could make to the OS to remedy this problem.

Priority inversion: higher-priority task1 is blocked by task3 holding s2, and then by task2 coming in between.

-> Solution: implement priority inheritance or priority ceiling protocol at least on s2, if not all semaphores. This would elevate priority of task3 while holding s2, such that it can complete and release s2 without task2 coming in between.

Name: _____

Problem 4 (20+5 points): Synchronization 2

Assuming an OS that implements spinlock semaphores and a strict priority scheduler (lower number is higher priority), observe the application code below:

```
void Task0(){ long sr;

    while(1) {

        OS_bWait(&s1);

        sr = StartCritical();

OS_bWait(&s1);

        // Toggle external LED
        // when data is updated
        GPIO_PORTF_DATA_R ^= 0x08;

        EndCritical(sr);

    }
}
```

```
void Task2(){

OS_bWait(&s1);

    // sample data and notify
    data = GPIO_PORTF_DATA_R;

    OS_bSignal(&s1);

    OS_bSignal(&s2);

OS_Kill();

}
```

```
void Task1(){

    while(1) {

OS_bWait(&s1); OS_bWait(&s2);

        // Print portf value
        // when data is updated
        printf("PF: %d\n\r", data);

        data = 0xDEADBEEF;

OS_bSignal(&s1);

    }

}
```

```
Sema4Type s1;  Sema4Type s2;
uint32_t data;
void main() {

    OS_Init(); // will initialize ports
    data = 0xDEADBEEF;
    OS_InitSemaphore(&s1, 0);  OS_InitSemaphore(&s2, 0);
    OS_AddThread(&Task0, 128, 1);
    OS_AddThread(&Task1, 128, 2);
    OS_AddPeriodicThread(&Task2, 1000, 2);
    OS_Launch();

}
```

Name: _____

There is at least one bug in the software. Identify all of bugs in the program and describe a potential solution that fixes them (including any followup issues). There may be less bugs than rows. Bonus points: extra credit for providing working code and fixes for the bugs as changes to the code above.

Bug	Explanation + Fix
<i>Task0 calls OS_bWait in a Start/EndCritical section.</i>	<i>A spinlock OS_Wait will enable interrupts, which will allow it to be interrupted, this breaking the critical section. -> Move OS_bWait() before StartCritical, or use as separate mutex semaphore to protect the critical section.</i>
<i>Task2 calls OS_bWait() from an interrupt</i>	<i>Task2 runs in a periodic timer interrupt context, and OS_bWait() in an interrupt locks up the system -> Remove OS_bWait() from Task2. This in turn creates a problem with s1 being signaled too often in Task1, so also need to remove OS_bSignal() from Task1</i>
<i>Deadlock on s1 in Task0</i>	<i>Task0 grabs s1 but never releases it. -> With change above, intended behavior is for Task2 to release s1, which in turn releases Task0 and Task1.</i>
<i>Task1 never runs</i>	<i>Task0 has higher priority and will always grab s1 before Task1 (fairness) -> Use two separate semaphores to signal Task1 and Task2</i>
<i>Task2 calls OS_Kill() from an interrupt</i>	<i>Task2 runs as a period background thread. OS_Kill() only works for foreground threads. -> Remove OS_Kill() from Task2</i>

Name: _____

Problem 5 (20 points): Scheduling

Given a 3-task system with the following characteristics (periods and minimum/maximum execution times):

Task	Execution Time	Period
T1	5ms $\pm$ 5ms	40ms
T2	15ms $\pm$ 5ms	60ms
T3	10ms $\pm$ 10ms	120ms

- a) Is this taskset schedulable to meet all deadlines (equal to periods) using a fixed-priority scheduler with a rate-monotonic scheduling (RMS) scheme?

Worst-case system utilization is $10/40 + 20/60 + 20/120 = 75\%$

This is below the RMS limit of $3(2^{1/3}-1) \approx 78\%$ for $n=3$ tasks that guarantees schedulability, i.e. this task set will meet all deadlines.

- b) What is the maximum, i.e. worst-case jitter (based on the start times of tasks) that can be experienced by each of the 3 tasks using an RMS strategy?

Task T1: highest priority, so will always run at arrival time, i.e. zero jitter

Task T2: can be delayed by task T1, which has a worst-case execution time of 10ms, i.e. worst-case jitter is 10ms (worst-case is experiencing the full delay in one iteration and none in the next).

Task T3: can be delayed by both tasks T1 and T2, with a combined worst-case delay of $10ms + 20ms = 30ms$ worst-case jitter.

Name: _____

- c) Can you come up with a non-preemptive fixed/static schedule, i.e. a schedule in which tasks start at fixed, predetermined times and can not be interrupted in the middle of their execution to meet all deadlines while minimizing worst-case jitter among all tasks? Draw your schedule for one hyper-period. What is the worst-case jitter for each task?

There are multiple possible solutions.

See one schedule below, based on worst-case execution times of each task.

Task T2 experiences a jitter of 10ms, all other tasks have zero jitter.

Alternative is to let T2 run before T1 in the beginning, then T1 would experience a jitter of 10ms and T2 none.

